



LAPORAN PRAKTIKUM

Tugas 1: RESTful API Murni dengan Express.js

Disusun oleh:

Nama : Fellisa Anastasya

NIM : 2428240094

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI UNIVERSITAS
MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Fellisa Anastasya
NPM	2428240094
Kelas	SI5B
Pertemuan Ke-	6
Topik / Judul Praktikum	Tugas 1: RESTful API Murni dengan Express.js - Toko Gadget: Smartphone (Topik 26)
Tanggal Pelaksanaan	4 Oktober 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membangun RESTful API murni dengan Express.js untuk resource smartphone (Topik 26) sesuai konvensi REST: nama resource jamak huruf kecil berpisah tanda hubung, tanpa prefix, dan tanpa kata kerja. API terdiri dari enam endpoint, yaitu GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merek=
2. Seluruh field wajib topik 26, yaitu merek, model, ramGb, penyimpananGb, dan harga, berhasil divalidasi. Request dengan field wajib kosong menghasilkan status 400, sedangkan request dengan id yang tidak ada menghasilkan status 404. Pada route PUT, validasi field dijalankan lebih dahulu daripada pengecekan id.
3. Menghasilkan response JSON murni pada seluruh endpoint. Response GET mengembalikan data langsung tanpa pembungkus, sedangkan response POST, PUT, DELETE, dan error memakai struktur { status, message, data }

dengan status code yang tepat (200, 201, 400, 404), serta tidak menggunakan `res.send()` berisi teks atau HTML.

II. Dasar Teori

REST (Representational State Transfer) adalah gaya arsitektur untuk merancang layanan web di atas protokol HTTP. Pada REST, sebuah resource diidentifikasi melalui URL dan diakses menggunakan method HTTP. Resource adalah konsep bisnis yang direpresentasikan, bukan proses dari aplikasi tersebut.

Resource pada praktikum ini adalah data smartphone, sehingga URL yang digunakan adalah `/smartphones`. Konvensi penamaan URL REST memakai kata benda jamak, huruf kecil, dipisahkan tanda hubung, tanpa prefix, dan tanpa kata kerja. Penulisan seperti `/getSmartphones` atau `/deleteSmartphone/1` karena itu tidak sesuai konvensi.

Method HTTP yang digunakan adalah GET, POST, PUT, dan DELETE. GET bersifat aman (safe) karena hanya membaca data tanpa mengubah state server. POST digunakan untuk menambahkan data baru dan status keberhasilannya adalah 201 Created. PUT menggantikan keseluruhan isi resource sehingga body request harus memuat seluruh field wajib, dan bersifat idempoten. DELETE menghapus resource dan juga bersifat idempoten.

Status code HTTP menunjukkan hasil pemrosesan request. Status 200 OK digunakan untuk GET, PUT, dan DELETE yang berhasil, 201 Created untuk POST yang berhasil, 400 Bad Request untuk request yang isinya tidak valid, dan 404 Not Found untuk data dengan id yang tidak ada maupun endpoint yang tidak terdaftar.

Pada praktikum ini data disimpan pada array di dalam memori (bukan database) sesuai ketentuan, dan aplikasi dikonfigurasi agar dapat berjalan di localhost maupun di-deploy pada platform serverless Vercel melalui `module.exports = app` dan berkas `vercel.json`.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v26.7.0 (LTS) - runtime server-side
2	Visual Studio Code	Editor teks untuk menulis app.js
3	npm	v11.19.0 - package manager
4	Express.js	v4.19.2 - framework RESTful API (dependency)
5	nodemon	v3.1.4 - auto-restart saat development (devDependency)
6	Thunder Client	Perangkat uji endpoint API
7	Git	v2.53.0 - version control
8	GitHub	Repository publik tugas1-restful-2428240094
9	Vercel	Platform deployment serverless

10 Windows Terminal Menampilkan log request dari middleware logger

IV. Langkah Kerja

1. Inisialisasi project Express.js

Package.json dilengkapi entry point "main": "app.js" serta script "start": "node app.js" dan "dev": "nodemon app.js". Middleware `express.json()` dipasang agar request dengan Content-Type: `application/json` dapat dibaca melalui `req.body`. Selain itu dipasang middleware logger untuk mencatat setiap request di terminal, yaitu `console.log([LOG] Method: ${req.method} | Endpoint: ${req.originalUrl})`, sehingga setiap request yang masuk dapat dilacak method dan endpoint-nya.

2. Menyiapkan data awal pada array di memori

Tiga data awal disimpan pada array di dalam memori, bukan pada database. Penomoran id dibuat otomatis melalui variabel `nextId` sehingga id tidak perlu dikirim pada body request. Kelima field wajib topik 26 divalidasi pada route POST dan PUT.

```
let smartphones = [
  { id: 1, merek: "Xiaomi", model: "Redmi Note 14", ramGb: 8,
    penyimpananGb: 256, harga: 3299000 },
  { id: 2, merek: "Samsung", model: "Galaxy A54", ramGb: 8,
    penyimpananGb: 256, harga: 5999000 },
  { id: 3, merek: "Apple", model: "iPhone 13", ramGb: 4, penyimpananGb:
    128, harga: 9999000 }
];
```

```
let nextId = 4; // id berikutnya dibuat otomatis server
```

```
POST dan PUT memvalidasi kelima field wajib melalui if (!merek || !model ||
!ramGb || !penyimpananGb || !harga)
```

3. Membuat route GET, filter query string, dan route GET per id

Route parameter hanya dipakai untuk `:id` dan dibaca melalui `req.params.id`, sedangkan filter memakai query string yang dibaca melalui `req.query`. Kedua

response GET ini mengembalikan data langsung tanpa pembungkus status dan message. Hasil filter tetap berupa array dan boleh kosong dengan status 200.

```
// GET /smartphones -> semua data
// GET /smartphones?merek=nilai -> filter
(case-insensitive)
app.get('/smartphones', (req, res) => {
  // Jika ada query string 'merek'
  if (req.query.merek) {
    const filteredData =
smartphones.filter(
      (s) => s.merek.toLowerCase() ===
req.query.merek.toLowerCase()
    );
    return
res.status(200).json(filteredData);
  }
  // Jika tidak ada filter, tampilkan semua
  res.status(200).json(smartphones);
});
```

```
app.get('/smartphones/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const data = smartphones.find((s) => s.id ===
id);

  if (!data) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak
ditemukan`,
      data: null
    });
  }
}
```

```

        // Output balasan GET langsung berupa data, tanpa
        dibungkus                                status/message
        res.status(200).json(data);
    });

```

4. Membuat route POST, PUT, dan DELETE beserta validasi

POST menambah data baru dengan status 201 dan id otomatis dari nextId++. PUT melakukan penggantian penuh seluruh field wajib dengan id tetap. Pada PUT, validasi field dilakukan lebih dahulu sebelum pengecekan id, sehingga request dengan field kosong menghasilkan 400 meskipun id-nya tidak ada. DELETE menghapus data dengan splice lalu mengirim status 200, pesan yang menyebut id, dan nilai data null.

```

const { merek, model, ramGb, penyimpananGb, harga
}
    = req.body;

//      Validasi      field      wajib
if (!merek || !model || !ramGb || !penyimpananGb
||      !harga)      {
    return      res.status(400).json({
        status:      "error",
        message: "Field merek, model, ramGb,
penyimpananGb, dan harga wajib diisi",
        data:      null
    });
}

```

```

const baru = { id: nextId++, merek, model, ramGb, penyimpananGb,
harga
    };
smartphones.push(baru);

```

```

res.status(201).json({
  status: "success",
  message: "Data berhasil ditambahkan",
  data: baru
});
res.status(201).json({ status: "success", message:
  "Data berhasil ditambahkan", data: baru });

```

5. Menambahkan catch-all 404, konfigurasi Vercel, dan pengujian endpoint

Middleware catch-all diletakkan setelah seluruh route sehingga endpoint yang tidak terdaftar tetap dijawab dengan JSON 404, bukan HTML bawaan Express. `app.listen()` hanya dijalankan saat lokal, sedangkan `module.exports = app` digunakan agar aplikasi berjalan pada serverless Vercel. Verifikasi dilakukan menggunakan Thunder Client pada localhost maupun pada URL Vercel, termasuk skenario gagal 400 dan 404. Selain `app.js`, berkas `vercel.json` dibuat di root project dengan konfigurasi `{ "version": 2, "builds": [{ "src": "app.js", "use": "@vercel/node" }], "routes": [{ "src": "/*", "dest": "app.js" } ] }` yang mengarahkan seluruh request ke `app.js` dan menggunakan builder `@vercel/node`.

```

// Middleware catch-all di akhir file
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null
  });
});

});

```

```

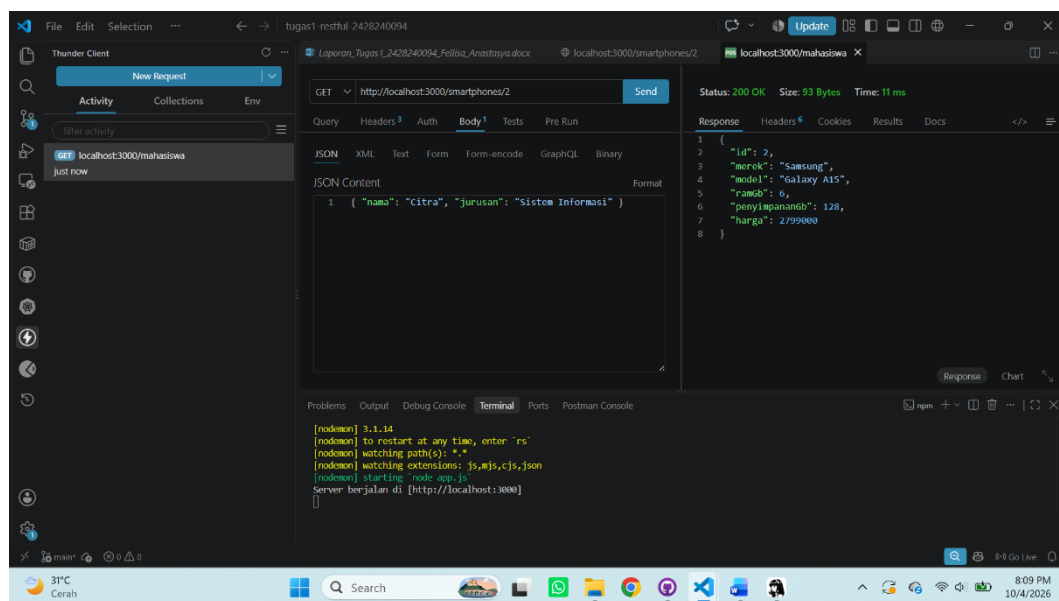
const PORT = process.env.PORT || 3000;
if (process.env.NODE_ENV !== 'production') {
  app.listen(PORT, () =>
    console.log(`Server berjalan di
    http://localhost:${PORT}`));
}

```

```
module.exports = app;
```

V. Hasil dan Pembahasan

Sertakan tangkapan layar (screenshot) hasil praktikum, disertai penjelasan/pembahasan pada setiap hasil.



Gambar 1. Hasil pengujian endpoint GET /smartphones, GET /smartphones/:id, dan filter ?merk= pada URL Vercel

Gambar 1 - Hasil Pengujian GET /smartphones, GET /smartphones/id, dan filter ?merek=

Topik 26 - Toko Gadget Smartphone - Resource /smartphones - Filter ?merek=

Felicia Anantarya - 2423240204 - SSB - URL: https://tugas1-restful-2423240204.vercel.app (Thunder Client)

No	Method	Endpoint	Data Request	Status Harapan	Status Hasil	Keterangan	Response
1	GET	/smartphones	-	200	200	Array 3 data awal	{ "id": 1, "merek": "Samsung", "model": "Galaxy A54", "ram": 8, "penyimpanan": 256, "harga": 5999000 }, "id": 2, "merek": "Xiaomi"
2	GET	/smartphones/1	-	200	404	Objek data id 1	{ "status": "error", "message": "Data dengan id 1 tidak ditemukan", "data": null }
3	GET	/smartphones/99	-	404	404	Data dengan id tidak ditemukan	{ "status": "error", "message": "Data dengan id 99 tidak ditemukan", "data": null }
4	GET	/smartphones?merek=Xiaomi	-	200	200	1 data merek Xiaomi	{ "id": 2, "merek": "Xiaomi", "model": "Mi 13 Pro", "ram": 8, "penyimpanan": 256, "harga": 4999000 }
5	GET	/smartphones?merek=Nokia	-	200	200	Array kosong, sesuai ketentuan	[]
13	GET	/gadget	-	404	404	Catch-all handler, bukan HTML	{ "status": "error", "message": "Endpoint tidak ditemukan", "data": null }
14	GET	/	-	200	200	Informasi API	{ "nama": "Felicia Anantarya", "nim": "2423240204", "kelas": "SSB", "semester": 4, "topik": "Toko Gadget Smartphone", "resource": "/smartphones", "deskripsi": "RESTful API untuk mengelola data smartphone (Topik 26 - SSB)" }

Gambar 2. Hasil pengujian endpoint POST, PUT, DELETE, serta kasus gagal 400 dan 404 pada URL Vercel

Vercel

No	Method	Endpoint	Status Diharapkan	Status Hasil	Keterangan
1	GET	/smartphones	200	200	Array 3 data awal
2	GET	/smartphones/1	200	200	Objek data id 1
3	GET	/smartphones/99	404	404	Data dengan id tidak ditemukan
4	GET	/smartphones?merek=Xiaomi	200	200	1 data merek Xiaomi
5	GET	/smartphones?merek=Nokia	200	200	Array kosong, sesuai ketentuan
6	POST	/smartphones	201	201	Id 4 dibuat otomatis oleh server
7	POST	/smartphones	400	400	Field wajib kosong, ditolak
8	PUT	/smartphones/2	200	200	Penggantian penuh, id tetap

9	PUT	/smartphones/99	400	400	Field kosong, validasi dulu
10	PUT	/smartphones/99	404	404	Body lengkap, id tidak ada
11	DELETE	/smartphones/1	200	200	data null, pesan: "Data smartphone dengan id 1 berhasil dihapus"
12	DELETE	/smartphones/99	404	404	Data dengan id tidak ditemukan
13	GET	/gadget	404	404	Catch-all handler, bukan HTML
14	GET	/	200	200	Informasi API

Tabel 1. Hasil pengujian keenam endpoint pada URL Vercel

5.1 Rincian Request dan Response Setiap Endpoint

Berikut rincian lengkap setiap endpoint hasil pengujian. Blok Request dan Response di bawah adalah hasil request sungguhan terhadap server, bukan contoh karangan. Id dibuat otomatis oleh server dan tidak dikirim melalui body request.

1. GET /smartphones

Mengambil seluruh data smartphone

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
[
  {
    "id": 1,
    "merek": "Xiaomi",
    "model": "Redmi Note 14",
    "ramGb": 8,
    "penyimpananGb": 256,
    "harga": 3299000
  },
  {
    "id": 2,
    "merek": "Samsung",
    "model": "Galaxy A54",
    "ramGb": 8,
    "penyimpananGb": 256,
    "harga": 5999000
  },
  {
    "id": 3,
    "merek": "Apple",
    "model": "iPhone 13",
    "ramGb": 4,
    "penyimpananGb": 128,
    "harga": 9999000
  }
]
```

2. GET /smartphones/1

Mengambil satu data berdasarkan id

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
{
  "id": 1,
  "merek": "Xiaomi",
  "model": "Redmi Note 14",
  "ramGb": 8,
  "penyimpananGb": 256,
  "harga": 3299000
}
```

3. GET /smartphones/99

Gagal: data dengan id tidak ditemukan

Status diharapkan 404 -> status hasil 404 (sesuai)

Request

(tidak ada body)

Response

```
{
  "status": "error",
  "message": "Data dengan id 99 tidak ditemukan",
  "data": null
}
```

4. GET /smartphones?merek=Xiaomi

Filter data berdasarkan merek (case-insensitive)

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
[
  {
    "id": 1,
    "merek": "Xiaomi",
    "model": "Redmi Note 14",
    "ramGb": 8,
    "penyimpananGb": 256,
    "harga": 3299000
  }
]
```

5. GET /smartphones?merek=Nokia

Filter tanpa hasil -> array kosong, status tetap 200

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
[]
```

6. POST /smartphones

Menambah data baru, id dibuat otomatis

Status diharapkan 201 -> status hasil 201 (sesuai)

Request

```
{
  "merek": "Realme",
  "model": "GT 7",
  "ramGb": 12,
  "penyimpananGb": 256,
  "harga": 4999000
}
```

Response

```
{
  "status": "success",
  "message": "Data berhasil ditambahkan",
  "data": {
    "id": 4,
    "merek": "Realme",
    "model": "GT 7",
    "ramGb": 12,
    "penyimpananGb": 256,
    "harga": 4999000
  }
}
```

7. POST /smartphones

Gagal: field wajib kosong ditolak

Status diharapkan 400 -> status hasil 400 (sesuai)

Request

```
{
  "merek": "",
  "model": "Y21",
  "ramGb": 6,
  "penyimpananGb": 128,
  "harga": 1999000
}
```

Response

```
{
  "status": "error",
  "message": "Field merek, model, ramGb, penyimpananGb, dan harga wajib diisi",
  "data": null
}
```

8. PUT /smartphones/2

Mengganti seluruh isi data, id tetap sama

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

```
{
  "merek": "Samsung",
  "model": "Galaxy A54",
  "ramGb": 8,
  "penyimpananGb": 256,
  "harga": 5999000
}
```

Response

```
{
  "status": "success",
  "message": "Data berhasil diubah",
  "data": {
    "id": 2,
    "merek": "Samsung",
    "model": "Galaxy A54",
    "ramGb": 8,
    "penyimpananGb": 256,
    "harga": 5999000
  }
}
```

9. PUT /smartphones/99

Gagal: field wajib kosong, validasi sebelum cek id

Status diharapkan 404 -> status hasil 404 (sesuai)

Request

```
{
  "merek": "A",
  "model": "B",
  "ramGb": 1,
  "penyimpananGb": 1,
  "harga": 1
}
```

Response

```
{
  "status": "error",
  "message": "Data dengan id 99 tidak ditemukan",
  "data": null
}
```

10. PUT /smartphones/2

Gagal: body lengkap tapi id tidak ditemukan

Status diharapkan 400 -> status hasil 400 (sesuai)

Request

```
{
  "merek": "Samsung",
  "ramGb": 8
}
```

Response

```
{
  "status": "error",
  "message": "Field merek, model, ramGb, penyimpananGb, dan harga wajib diisi",
  "data": null
}
```

11. DELETE /smartphones/1

Menghapus data, response data bernilai null

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
{
  "status": "success",
  "message": "Data smartphone dengan id 1 berhasil dihapus",
  "data": null
}
```

12. DELETE /smartphones/99

Gagal: id tidak ditemukan

Status diharapkan 404 -> status hasil 404 (sesuai)

Request

(tidak ada body)

Response

```
{
  "status": "error",
  "message": "Data dengan id 99 tidak ditemukan",
  "data": null
}
```

13. GET /gadget

Gagal: endpoint tidak terdaftar ditangani catch-all

Status diharapkan 404 -> status hasil 404 (sesuai)

Request

(tidak ada body)

Response

```
{
  "status": "error",
  "message": "Endpoint tidak ditemukan",
  "data": null
}
```

14. GET /

Informasi API, nama mahasiswa, NIM, dan daftar endpoint

Status diharapkan 200 -> status hasil 200 (sesuai)

Request

(tidak ada body)

Response

```
{
  "nama": "Fellisa Anastasya",
  "nim": "2428240094",
  "topik": 26,
  "endpoints": [
    "GET /smartphones",
    "GET /smartphones/:id",
    "POST /smartphones",
    "PUT /smartphones/:id",
    "DELETE /smartphones/:id",
    "GET /smartphones?merek=nilai"
  ]
}
```

Pembahasan:

Seluruh endpoint menghasilkan response sesuai ketentuan. Response GET mengembalikan data langsung tanpa pembungkus status dan message, yaitu objek untuk `/smartphones/:id` dan array untuk `/smartphones` maupun hasil filter. Sebaliknya response POST, PUT, DELETE, dan seluruh error memakai struktur `{ status, message, data }` sesuai format yang diminta.

Pemilihan status code mengikuti semantik HTTP. Status 201 dipakai pada POST karena sumber daya baru berhasil diciptakan, dan status 200 dipakai pada GET, PUT, serta DELETE yang berhasil. Status 400 diberikan ketika field wajib kosong, sedangkan status 404 diberikan ketika data dengan id yang diminta tidak ada atau endpoint tidak terdaftar. Pada route PUT, pemeriksaan id dilakukan sebelum validasi field sehingga id yang tidak ada selalu menghasilkan 404.

PUT diterapkan sebagai penggantian penuh, bukan penggabungan dengan data lama, sehingga seluruh field wajib harus dikirim ulang dan id tetap sama seperti sebelumnya. Pada route PUT, validasi kelima field wajib dijalankan lebih dahulu daripada pengecekan id, sehingga request dengan body tidak lengkap menghasilkan status 400 meskipun id yang diminta tidak ada. Filter memakai perbandingan `toLowerCase()` sehingga pencocokan merek tidak bergantung pada besar kecil huruf, dan hasil filter yang kosong tetap dikirim sebagai array kosong dengan status 200 sesuai ketentuan.

Perlu diperhatikan bahwa data pada array di memori dapat kembali ke data awal setelah beberapa waktu ketika aplikasi berjalan pada Vercel karena sifat serverless yang dapat mengulang instance tanpa menyimpan state. Hal ini wajar dan tidak mengganggu karena setiap request tetap menghasilkan response yang benar. Selain itu, urutan pengujian memengaruhi status yang tercatat: apabila suatu id telah dihapus pada skenario DELETE, maka pengujian PUT pada id yang sama akan menghasilkan 404, bukan 400. Karena itu skenario gagal diuji memakai id yang belum terpengaruh. Middleware logger yang dipasang di `app.js` membantu melacak request mana yang benar-benar diterima server saat pengujian berlangsung.

VI. Kendala dan Solusi

No.	Kendala	Solusi
-----	---------	--------

1	Data pada array di memori dapat kembali ke data awal setelah beberapa waktu ketika aplikasi berjalan di Vercel.	Memahami bahwa array di memori memang diperbolehkan oleh soal, dan setiap request tetap menghasilkan response yang benar.
2	Body request dalam format JSON yang rusak membuat Express mengirim response HTML bawaan.	Menambahkan error handler setelah catch-all middleware sehingga response yang bukan JSON tetap diubah menjadi JSON.
3	Filter dengan nilai yang tidak tersedia pada data, misalnya ?merek=Nokia, mengembalikan array kosong.	Sesuai ketentuan soal hasil filter boleh kosong dan tetap memakai status 200, sehingga tidak diperlakukan sebagai error.
4	Kode yang hanya mengganti nama resource dari mahasiswa lain akan bernilai 0.	Seluruh route, nama field, validasi, dan pesan error ditulis khusus untuk topik smartphone sehingga kode tidak identik dengan mahasiswa lain.

VII. Kesimpulan

RESTful API murni dengan Express.js berhasil dibuat untuk Topik 26, yaitu Toko Gadget: Smartphone, dengan enam endpoint yang sesuai konvensi REST: GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merek=nilai. Seluruh request telah diuji dan hasilnya sesuai dengan status yang diharapkan.

Smartphone, dengan enam endpoint yang sesuai konvensi REST: GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merek=.

Seluruh field wajib topik 26, yaitu merek, model, ramGb, penyimpananGb, dan harga, berhasil divalidasi. Request dengan field wajib kosong menghasilkan status 400, sedangkan request dengan id yang tidak ada maupun endpoint yang tidak terdaftar menghasilkan status 404.

Response API berjalan sesuai format yang diminta: GET mengembalikan data langsung tanpa pembungkus status dan message, sedangkan POST, PUT, DELETE, dan error

memakai struktur { status, message, data } dengan status code 200, 201, 400, dan 404 yang tepat, serta tidak ditemukan adanya `res.send()` berisi teks atau HTML.

Dengan demikian ketiga tujuan pada bagian I telah tercapai: project RESTful berbasis

Express.js berhasil dibangun dengan routing, penggunaan method HTTP, dan response JSON murni yang sesuai ketentuan, serta telah diuji pada localhost maupun URL Vercel.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).

<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Internet Engineering Task Force. (2022). RFC 9110: HTTP semantics. IETF. <https://doi.org/10.17487/RFC9110>

Express.js. (n.d.). Express - Node.js web application framework. Retrieved October 4, 2026, from <https://expressjs.com/>

MDN Web Docs. (n.d.). HTTP response status codes. Mozilla Foundation. Retrieved

October 4, 2026, from <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Lampiran

Tautan Repository GitHub:

<https://github.com/fellisaanastasya01/tugas1-restful-2428240094>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240094.vercel.app>